Topic: Types of Linked List

Q. Elaborate on the different types of Linked Lists. Explain Singly, Circular, and Doubly Linked Lists with their structures and diagrams.

> 📌 **Definition to Remember**
> Linked Lists are classified based on how nodes are connected: **Singly Linked List** (one-way traversal), **Circular Linked List** (last node points to first, continuous circle), and **Doubly Linked List** (two-way traversal using previous and next pointers).

---

1. Singly Linked List

The simplest type. Each node contains data and a single pointer (`next`) pointing to the next node.
* **Traversal:** Unidirectional (forward only).
* **End Indicator:** The last node points to `NULL` .

Diagram:

2. Circular Linked List

The `next` pointer of the **last node** does NOT point to `NULL` . Instead, it points back to the **first node (head)**, forming a continuous circle.
* **Traversal:** Continuous — you can start at any node and traverse the entire list.
* **Usage:** Round-Robin scheduling in OS, multiplayer games (turn passing).

Diagram:

3. Doubly Linked List

Each node contains **three fields**: data, a pointer to the **next** node, and a pointer to the **previous** node.
* **Traversal:** Bidirectional (can move forward and backward).
* **Usage:** Browser History (forward/back), music player playlists.

Structure in C:

```
struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};
```

Diagram:

```
NULL ⟵ [Prev|Data|Next] ⟷ [Prev|Data|Next] ⟷ [Prev|Data|Next] ⟶ NULL
          (Head)
```

> ⭐ **Must-Write Points (for 10 marks)**
>
> 1. Three primary types: **Singly**, **Circular**, and **Doubly** Linked Lists.
> 2. **Singly Linked List**: contains `data` and `next` pointer; unidirectional traversal; ends with `NULL`.
> 3. **Circular Linked List**: last node points back to the `head`; continuous traversal; no `NULL` at the end.
> 4. Circular lists are used in Round-Robin CPU scheduling.
> 5. **Doubly Linked List**: contains `prev`, `data`, and `next` pointers; bidirectional traversal.
> 6. Doubly lists use more memory per node but allow moving backwards easily.
> 7. Doubly lists are used in browser history (back/forward functionality).

> ⚡ **Quick Recall**
>
> `Singly (Forward only, ends in NULL) → Circular (Last points to Head, continuous loop) → Doubly (prev + next pointers, bidirectional, uses more memory)`